

[pd] Lecture de fichiers – read_*

DESCRIPTION : Charge des données depuis des fichiers ou sources externes en DataFrame
POURQUOI : Point d'entrée principal de pandas. Supporte des dizaines de formats.
CONTEXTE : Import de données, ETL, pipelines de traitement.

— CSV / TEXTE —

FONCTIONS PRINCIPALES :

pd.read_csv("f.csv")	Lit un fichier CSV
pd.read_csv("f.csv", sep=";")	Séparateur personnalisé
pd.read_csv("f.csv", sep="\t")	Séparateur tabulation
pd.read_csv("f.csv", delimiter=" ")	Alias de sep
pd.read_csv("f.csv", header=None)	Sans ligne d'en-tête
pd.read_csv("f.csv", header=0)	En-tête à la ligne 0 (défaut)
pd.read_csv("f.csv", header=[0,1])	En-tête multi-niveaux (MultiIndex)
pd.read_csv("f.csv", names=["a","b"])	Noms de colonnes personnalisés
pd.read_csv("f.csv", index_col=0)	Première colonne comme index
pd.read_csv("f.csv", index_col="col")	Colonne nommée comme index
pd.read_csv("f.csv", index_col=[0,1])	Multi-index depuis colonnes
pd.read_csv("f.csv", usecols=["a","b"])	Colonnes spécifiques seulement
pd.read_csv("f.csv", usecols=[0,2])	Colonnes par position
pd.read_csv("f.csv", usecols=lambda c: c.startswith("val"))	Par condition
pd.read_csv("f.csv", dtype={"col": int})	Types forcés
pd.read_csv("f.csv", dtype=str)	Tout lire en chaîne
pd.read_csv("f.csv", nrows=100)	Lit seulement N lignes
pd.read_csv("f.csv", skiprows=5)	Saute les N premières lignes
pd.read_csv("f.csv", skiprows=[1,3,5])	Saute des lignes spécifiques
pd.read_csv("f.csv", skiprows=lambda i: i%2)	Saute une ligne sur deux
pd.read_csv("f.csv", skipfooter=2)	Saute les N dernières lignes
pd.read_csv("f.csv", chunksize=1000)	Lit par chunks (gros fichiers)
pd.read_csv("f.csv", compression="gzip")	Lit un fichier .gz compressé
pd.read_csv("f.csv", compression="zip")	Lit un fichier .zip
pd.read_csv("f.csv", compression="bz2")	Lit un fichier .bz2
pd.read_csv("f.csv", compression="xz")	Lit un fichier .xz
pd.read_csv("f.csv", compression="zstd")	Lit un fichier .zst
pd.read_csv("f.csv", encoding="utf-8")	Encodage UTF-8
pd.read_csv("f.csv", encoding="latin-1")	Encodage latin-1 / ISO-8859-1
pd.read_csv("f.csv", encoding="utf-8-sig")	UTF-8 avec BOM (fichiers Windows)
pd.read_csv("f.csv", encoding_errors="replace")	Remplace les erreurs d'encodage
pd.read_csv("f.csv", na_values=["N/A","-"])	Valeurs considérées comme NaN
pd.read_csv("f.csv", keep_default_na=False)	Ne pas inférer les NaN par défaut
pd.read_csv("f.csv", parse_dates=["date"])	Parse une colonne en datetime
pd.read_csv("f.csv", parse_dates=[["d","h"]])	Combine colonnes date + heure
pd.read_csv("f.csv", dayfirst=True)	Format JJ/MM/AAAA
pd.read_csv("f.csv", on_bad_lines="skip")	Ignore les lignes malformées
pd.read_csv("f.csv", on_bad_lines="warn")	Avertit sur les lignes malformées
pd.read_csv("f.csv", comment="#")	Ignore les lignes de commentaires
pd.read_csv("f.csv", thousands=",")	Séparateur de milliers
pd.read_csv("f.csv", decimal=",")	Séparateur décimal (ex: français)
pd.read_csv("f.csv", quotechar='"')	Caractère de citation
pd.read_csv("f.csv", escapechar="\\"')	Caractère d'échappement
pd.read_csv("f.csv", quoting=0)	Mode citation (0=MINIMAL)
pd.read_csv("f.csv", doublequote=True)	Double quote comme échappement
pd.read_csv("f.csv", low_memory=False)	Désactive inférence de type mixte
pd.read_csv("f.csv", memory_map=True)	Utilise memory-mapped I/O
pd.read_csv("f.csv", float_precision="high")	Précision flottante haute
pd.read_csv("f.csv", true_values=["oui"])	Valeurs interprétées comme True
pd.read_csv("f.csv", false_values=["non"])	Valeurs interprétées comme False
pd.read_csv("f.csv", verbose=True)	Affiche les infos de lecture
pd.read_csv(io.StringIO(texte))	Lit depuis une chaîne en mémoire
pd.read_csv(io.BytesIO(bytes_data))	Lit depuis des bytes en mémoire
pd.read_csv(url)	Lit depuis une URL HTTP/FTP/S3
pd.read_table("f.tsv")	Lit un fichier délimité (sep=\t)
pd.read_table("f.txt", sep="\s+")	Séparateur espace(s)
pd.read_fwf("f.txt")	Lit un fichier à largeur de colonnes fixe
pd.read_fwf("f.txt", widths=[10,5,8])	Largeurs de colonnes spécifiées
pd.read_fwf("f.txt", colspecs=[(0,10),(10,15)])	Positions de colonnes
pd.read_clipboard()	Lit depuis le presse-papiers
pd.read_clipboard(sep="\s+")	Presse-papiers avec séparateur

— EXCEL —

<code>pd.read_excel("f.xlsx")</code>	Lit la première feuille
<code>pd.read_excel("f.xlsx", sheet_name="Sheet1")</code>	Feuille par nom
<code>pd.read_excel("f.xlsx", sheet_name=0)</code>	Feuille par index (0 = première)
<code>pd.read_excel("f.xlsx", sheet_name=2)</code>	Troisième feuille
<code>pd.read_excel("f.xlsx", sheet_name=[0,1])</code>	Plusieurs feuilles → dict de df
<code>pd.read_excel("f.xlsx", sheet_name=["A","B"])</code>	Feuilles par nom → dict de df
<code>pd.read_excel("f.xlsx", sheet_name=None)</code>	Toutes les feuilles → dict de df
<code>pd.read_excel("f.xlsx", header=0)</code>	Ligne d'en-tête (défaut=0)
<code>pd.read_excel("f.xlsx", header=None)</code>	Sans en-tête
<code>pd.read_excel("f.xlsx", header=[0,1])</code>	En-tête multi-niveaux (MultiIndex)
<code>pd.read_excel("f.xlsx", index_col=0)</code>	Première colonne comme index
<code>pd.read_excel("f.xlsx", index_col=[0,1])</code>	Multi-index depuis colonnes
<code>pd.read_excel("f.xlsx", usecols="A:D")</code>	Colonnes A à D (notation Excel)
<code>pd.read_excel("f.xlsx", usecols="A,C,E")</code>	Colonnes spécifiques (notation Excel)
<code>pd.read_excel("f.xlsx", usecols=[0,2,4])</code>	Colonnes par index
<code>pd.read_excel("f.xlsx", usecols=["Nom","Age"])</code>	Colonnes par nom
<code>pd.read_excel("f.xlsx", nrows=100)</code>	Lit seulement N lignes
<code>pd.read_excel("f.xlsx", skiprows=2)</code>	Saute les N premières lignes
<code>pd.read_excel("f.xlsx", skiprows=[1,3])</code>	Saute des lignes spécifiques
<code>pd.read_excel("f.xlsx", skipfooter=1)</code>	Saute les N dernières lignes
<code>pd.read_excel("f.xlsx", dtype={"col": str})</code>	Forcer le type d'une colonne
<code>pd.read_excel("f.xlsx", dtype=str)</code>	Tout lire en chaîne
<code>pd.read_excel("f.xlsx", na_values=["N/A"])</code>	Valeurs NaN personnalisées
<code>pd.read_excel("f.xlsx", keep_default_na=False)</code>	Ne pas inférer NaN
<code>pd.read_excel("f.xlsx", parse_dates=["date"])</code>	Convertit une colonne en datetime
<code>pd.read_excel("f.xlsx", parse_dates=[["d","h"]])</code>	Combine colonnes date + heure
<code>pd.read_excel("f.xlsx", thousands=",")</code>	Séparateur de milliers
<code>pd.read_excel("f.xlsx", decimal=",")</code>	Séparateur décimal
<code>pd.read_excel("f.xlsx", true_values=["oui"])</code>	Valeurs interprétées comme True
<code>pd.read_excel("f.xlsx", false_values=["non"])</code>	Valeurs interprétées comme False
<code>pd.read_excel("f.xlsx", comment="#")</code>	Ignore les lignes de commentaires
<code>pd.read_excel("f.xlsx", names=["a","b"])</code>	Renomme les colonnes à la lecture
<code>pd.read_excel("f.xlsx", converters={"col": str.strip})</code>	Convertisseurs par colonne
<code>pd.read_excel("f.xlsx", verbose=True)</code>	Affiche infos de lecture

EXEMPLES (EXCEL) :

```
# Lire une seule feuille
df = pd.read_excel("rapport.xlsx", sheet_name="Ventes", usecols="A:D")
# Lire toutes les feuilles dans un dict
dfs = pd.read_excel("rapport.xlsx", sheet_name=None)
df_ventes = dfs["Ventes"]
df_clients = dfs["Clients"]
# Fusionner toutes les feuilles en un seul DataFrame
dfs = pd.read_excel("rapport.xlsx", sheet_name=None)
df_all = pd.concat(dfs.values(), ignore_index=True)
# Lire plusieurs feuilles nommées
dfs = pd.read_excel("rapport.xlsx", sheet_name=["Jan","Fév","Mar"])
df_q1 = pd.concat(dfs.values(), ignore_index=True)
# En-tête multi-niveaux
df = pd.read_excel("rapport.xlsx", header=[0,1]) # MultiIndex de colonnes
```

— JSON —

<code>pd.read_json("f.json")</code>	Lit un fichier JSON
<code>pd.read_json("f.json", orient="records")</code>	Liste d'objets [{"a":1},{a":2}]
<code>pd.read_json("f.json", orient="index")</code>	Dict de dicts {"0":{"a":1}}
<code>pd.read_json("f.json", orient="columns")</code>	Dict de colonnes {"a":{"0":1}}
<code>pd.read_json("f.json", orient="values")</code>	Tableau de valeurs [[1,2],[3,4]]
<code>pd.read_json("f.json", orient="split")</code>	Dict avec keys/index/data
<code>pd.read_json("f.json", orient="table")</code>	Format JSON Table Schema
<code>pd.read_json("f.json", lines=True)</code>	Lit le format JSONL (une ligne = un objet)
<code>pd.read_json("f.json", dtype={"col": str})</code>	Types forcés
<code>pd.read_json("f.json", encoding="utf-8")</code>	Encodage spécifique
<code>pd.read_json("f.json", chunksize=1000)</code>	Lit par chunks (avec lines=True)
<code>pd.read_json("f.json", nrows=100)</code>	Lit N lignes (avec lines=True)
<code>pd.read_json("f.json", convert_dates=True)</code>	Convertit les dates automatiquement
<code>pd.read_json("f.json", convert_dates=["date"])</code>	Convertit colonnes spécifiques
<code>pd.read_json("f.json", keep_default_dates=False)</code>	Désactive la conversion auto
<code>pd.read_json("f.json", precise_float=True)</code>	Précision flottante maximale
<code>pd.read_json(url)</code>	Lit depuis une URL
<code>pd.json_normalize(data)</code>	Normalise un JSON imbriqué en df
<code>pd.json_normalize(data, record_path="items")</code>	Normalise un sous-champ liste
<code>pd.json_normalize(data, meta=["id","date"])</code>	Conserve des champs meta
<code>pd.json_normalize(data, max_level=2)</code>	Profondeur maximale d'expansion
<code>pd.json_normalize(data, sep=".")</code>	Séparateur pour les clés imbriquées

PARQUET	
<code>pd.read_parquet("f.parquet")</code>	Lit un fichier Parquet
<code>pd.read_parquet("f.parquet", columns=["a"])</code>	Colonnes spécifiques seulement
<code>pd.read_parquet("f.parquet", filters=...)</code>	Filtre lors de la lecture (pushdown)
<code>pd.read_parquet("dossier/")</code>	Lit un dossier Parquet partitionné
<code>pd.read_parquet("f.parquet", dtype_backend="numpy_nullable")</code>	Types nullable
<code>pd.read_parquet("f.parquet", dtype_backend="pyarrow")</code>	Types PyArrow
FEATHER	
<code>pd.read_feather("f.feather")</code>	Lit un fichier Feather
<code>pd.read_feather("f.feather", columns=["a"])</code>	Colonnes spécifiques
<code>pd.read_feather("f.feather", use_threads=True)</code>	Lecture multi-thread
HDF5	
<code>pd.read_hdf("f.h5", key="data")</code>	Lit un fichier HDF5
<code>pd.read_hdf("f.h5", key="/groupe/data")</code>	Chemin de clé imbriqué
<code>pd.read_hdf("f.h5", where="col > 5")</code>	Filtre lors de la lecture
<code>pd.read_hdf("f.h5", columns=["a","b"])</code>	Colonnes spécifiques
<code>pd.read_hdf("f.h5", start=0, stop=1000)</code>	Plage de lignes
PICKLE	
<code>pd.read_pickle("f.pkl")</code>	Lit un fichier Pickle
<code>pd.read_pickle("f.pkl", compression="gzip")</code>	Lit un pickle compressé
SQL	
<code>pd.read_sql("SELECT * FROM t", conn)</code>	Requête ou nom de table
<code>pd.read_sql_query("SELECT ...", conn)</code>	Requête SQL explicite
<code>pd.read_sql_query("SELECT ...", conn, index_col="id")</code>	Index depuis colonne
<code>pd.read_sql_query("SELECT ...", conn, parse_dates=["date"])</code>	Parse dates
<code>pd.read_sql_query("SELECT ...", conn, chunksize=1000)</code>	Par chunks
<code>pd.read_sql_query("SELECT ...", conn, params={"id": 5})</code>	Paramètres liés
<code>pd.read_sql_table("ma_table", conn)</code>	Lit une table entière
<code>pd.read_sql_table("t", conn, schema="public")</code>	Schéma SQL spécifique
<code>pd.read_sql_table("t", conn, columns=["a"])</code>	Colonnes spécifiques
<code>pd.read_sql_table("t", conn, index_col="id")</code>	Index depuis colonne
AUTRES FORMATS	
<code>pd.read_html("url")</code>	Lit les tableaux HTML d'une page
<code>pd.read_html("url", match="Tableau")</code>	Filtre par texte dans la page
<code>pd.read_html("url", header=0)</code>	Ligne d'en-tête
<code>pd.read_html("url", index_col=0)</code>	Colonne comme index
<code>pd.read_html("url", na_values=["-"])</code>	Valeurs NaN
<code>pd.read_html("url", encoding="utf-8")</code>	Encodage
<code>pd.read_html("f.html")</code>	Depuis un fichier HTML local
<code>pd.read_xml("f.xml")</code>	Lit un fichier XML
<code>pd.read_xml("f.xml", xpath="//item")</code>	Nœuds XML ciblés par XPath
<code>pd.read_xml("f.xml", namespaces={"ns":"url"})</code>	Espaces de noms XML
<code>pd.read_xml("f.xml", elems_only=True)</code>	Éléments uniquement
<code>pd.read_xml("f.xml", attrs_only=True)</code>	Attributs uniquement
<code>pd.read_xml("f.xml", encoding="utf-8")</code>	Encodage
<code>pd.read_stata("f.dta")</code>	Lit un fichier Stata
<code>pd.read_stata("f.dta", columns=["a","b"])</code>	Colonnes spécifiques
<code>pd.read_stata("f.dta", chunksize=1000)</code>	Par chunks
<code>pd.read_stata("f.dta", convert_categoricals=False)</code>	Ne pas convertir catégories
<code>pd.read_sas("f.sas7bdat")</code>	Lit un fichier SAS
<code>pd.read_sas("f.xpt")</code>	Lit un fichier SAS Transport
<code>pd.read_sas("f.sas7bdat", chunksize=5000)</code>	Par chunks
<code>pd.read_sas("f.sas7bdat", encoding="utf-8")</code>	Encodage
<code>pd.read_spss("f.sav")</code>	Lit un fichier SPSS
<code>pd.read_spss("f.sav", usecols=["a","b"])</code>	Colonnes spécifiques
<code>pd.read_spss("f.sav", convert_categoricals=True)</code>	Convertit les catégories
<code>pd.read_orc("f.orc")</code>	Lit un fichier ORC
<code>pd.read_orc("f.orc", columns=["a","b"])</code>	Colonnes spécifiques

EXEMPLES GÉNÉRAUX :

```
df = pd.read_csv("data.csv", sep=";", encoding="utf-8", parse_dates=["date"])
for chunk in pd.read_csv("gros.csv", chunksize=10000):
    traiter(chunk)

df = pd.read_sql("SELECT * FROM clients WHERE actif=1", connexion)
data = pd.json_normalize(json_data, record_path="items", meta=["date","source"])
```

[df] Écriture de fichiers – to_*

DESCRIPTION : Exporte un DataFrame vers différents formats de fichiers
POURQUOI : Sauvegarde des résultats d'analyse, échange de données entre systèmes.
CONTEXTE : Export de rapports, pipelines de données, sauvegardes.

— CSV / TEXTE —

FONCTIONS PRINCIPALES :

<code>df.to_csv("f.csv")</code>	Exporte en CSV (avec index)
<code>df.to_csv("f.csv", index=False)</code>	Sans index
<code>df.to_csv("f.csv", sep=";")</code>	Séparateur personnalisé
<code>df.to_csv("f.csv", sep="\t")</code>	Séparateur tabulation
<code>df.to_csv("f.csv", encoding="utf-8")</code>	Encodage UTF-8
<code>df.to_csv("f.csv", encoding="utf-8-sig")</code>	UTF-8 avec BOM (pour Excel Windows)
<code>df.to_csv("f.csv", compression="gzip")</code>	Compresse en .gz automatiquement
<code>df.to_csv("f.csv", compression="zip")</code>	Compresse en .zip
<code>df.to_csv("f.csv", compression="bz2")</code>	Compresse en .bz2
<code>df.to_csv("f.csv", compression="xz")</code>	Compresse en .xz
<code>df.to_csv("f.csv", compression="zstd")</code>	Compresse en .zst
<code>df.to_csv("f.csv", compression={"method": "gzip", "compresslevel": 9})</code>	Options avancées
<code>df.to_csv("f.csv", mode="a", header=False)</code>	Ajoute à un fichier existant
<code>df.to_csv("f.csv", columns=["a", "b"])</code>	Exporte seulement certaines colonnes
<code>df.to_csv("f.csv", float_format="%.2f")</code>	Format des flottants
<code>df.to_csv("f.csv", date_format="%d/%m/%Y")</code>	Format des dates
<code>df.to_csv("f.csv", na_rep="N/A")</code>	Représentation des NaN
<code>df.to_csv("f.csv", header=True)</code>	Inclut l'en-tête (défaut)
<code>df.to_csv("f.csv", header=["a", "b"])</code>	Noms d'en-tête personnalisés
<code>df.to_csv("f.csv", quotechar="'")</code>	Caractère de citation
<code>df.to_csv("f.csv", quoting=2)</code>	Mode citation (2=NONNUMERIC)
<code>df.to_csv("f.csv", doublequote=True)</code>	Double quote comme échappement
<code>df.to_csv("f.csv", escapechar="\\")</code>	Caractère d'échappement
<code>df.to_csv("f.csv", lineterminator="\n")</code>	Fin de ligne
<code>df.to_csv(None)</code>	Retourne une chaîne (pas de fichier)
<code>df.to_csv(buffer)</code>	Écrit dans un buffer (StringIO, etc.)

— EXCEL —

<code>df.to_excel("f.xlsx")</code>	Exporte en Excel (avec index)
<code>df.to_excel("f.xlsx", index=False)</code>	Sans index
<code>df.to_excel("f.xlsx", sheet_name="Données")</code>	Nom de feuille personnalisé
<code>df.to_excel("f.xlsx", columns=["a", "b"])</code>	Colonnes spécifiques
<code>df.to_excel("f.xlsx", header=True)</code>	Avec en-tête (défaut)
<code>df.to_excel("f.xlsx", header=["X", "Y"])</code>	Noms d'en-tête personnalisés
<code>df.to_excel("f.xlsx", startrow=2)</code>	Commence à écrire à la ligne 2
<code>df.to_excel("f.xlsx", startcol=1)</code>	Commence à écrire à la colonne 1
<code>df.to_excel("f.xlsx", na_rep="")</code>	Représentation des NaN
<code>df.to_excel("f.xlsx", float_format="%.2f")</code>	Format des flottants
<code>df.to_excel("f.xlsx", freeze_panes=(1,0))</code>	Figé la première ligne
<code>df.to_excel("f.xlsx", freeze_panes=(0,1))</code>	Figé la première colonne
<code>df.to_excel("f.xlsx", freeze_panes=(1,1))</code>	Figé ligne ET colonne
<code>df.to_excel("f.xlsx", merge_cells=True)</code>	Fusionne les cellules MultiIndex
<code>df.to_excel("f.xlsx", inf_rep="Inf")</code>	Représentation des infinis
<code>df.to_excel(writer, sheet_name="S1")</code>	Via ExcelWriter (multi-feuilles)

— ÉCRITURE MULTI-FEUILLES AVEC ExcelWriter —

DESCRIPTION : `pd.ExcelWriter` est le moyen natif pandas d'écrire plusieurs DataFrames dans un seul fichier Excel, chacun dans sa propre feuille.

```
# Syntaxe de base (context manager recommandé)
with pd.ExcelWriter("rapport.xlsx") as writer:
    df1.to_excel(writer, sheet_name="Ventes")
    df2.to_excel(writer, sheet_name="Clients")
    df3.to_excel(writer, sheet_name="Produits")
```

```
# Sans context manager (nécessite writer.save() ou writer.close())
writer = pd.ExcelWriter("rapport.xlsx")
df1.to_excel(writer, sheet_name="Feuille1")
df2.to_excel(writer, sheet_name="Feuille2")
writer.close()
```

```
# Options de ExcelWriter
pd.ExcelWriter("f.xlsx", mode="w")          Créer / écraser (défaut)
pd.ExcelWriter("f.xlsx", mode="a")          Ajouter des feuilles à un fichier existant
pd.ExcelWriter("f.xlsx", if_sheet_exists="replace") Remplace la feuille si elle existe
pd.ExcelWriter("f.xlsx", if_sheet_exists="new")  Crée une nouvelle feuille
pd.ExcelWriter("f.xlsx", if_sheet_exists="overlay") Superpose les données
pd.ExcelWriter("f.xlsx", if_sheet_exists="error") Lève une erreur (défaut)
pd.ExcelWriter("f.xlsx", date_format="DD/MM/YYYY") Format de date
```

```
pd.ExcelWriter("f.xlsx", datetime_format="DD/MM/YYYY HH:MM") Format datetime
```

```
# Accès aux attributs de l'ExcelWriter
writer.sheets          Dict {nom_feuille: objet_feuille}
writer.book            Le classeur sous-jacent
writer.path            Chemin du fichier
```

EXEMPLES (EXCEL) :

```
# Export simple une feuille
df.to_excel("résultat.xlsx", sheet_name="Résultat", index=False,
            freeze_panes=(1,0))

# Export multi-feuilles avec options
with pd.ExcelWriter("rapport_mensuel.xlsx") as writer:
    df_ventes.to_excel(writer, sheet_name="Ventes", index=False)
    df_clients.to_excel(writer, sheet_name="Clients", index=False)
    df_stats.to_excel(writer, sheet_name="Statistiques", startrow=1)

# Ajouter une feuille à un fichier existant
with pd.ExcelWriter("rapport.xlsx", mode="a",
                    if_sheet_exists="replace") as writer:
    df_nouveau.to_excel(writer, sheet_name="NouvelleFeuille")

# Plusieurs df sur la même feuille, positionnés différemment
with pd.ExcelWriter("dashboard.xlsx") as writer:
    df_nord.to_excel(writer, sheet_name="Régions", startrow=0, startcol=0)
    df_sud.to_excel(writer, sheet_name="Régions", startrow=0, startcol=5)

# Boucle sur des groupes pour créer une feuille par groupe
with pd.ExcelWriter("par_region.xlsx") as writer:
    for region, groupe in df.groupby("région"):
        groupe.to_excel(writer, sheet_name=region, index=False)
```

— JSON —

<code>df.to_json("f.json")</code>	Exporte en JSON (orient=columns)
<code>df.to_json("f.json", orient="records")</code>	Liste d'objets [{"a":1},{a":2}]
<code>df.to_json("f.json", orient="index")</code>	Dict de dicts {"0":{"a":1}}
<code>df.to_json("f.json", orient="columns")</code>	Dict de colonnes (défaut)
<code>df.to_json("f.json", orient="values")</code>	Tableau de valeurs brutes
<code>df.to_json("f.json", orient="split")</code>	Dict avec keys/index/data
<code>df.to_json("f.json", orient="table")</code>	JSON Table Schema
<code>df.to_json("f.json", indent=2)</code>	JSON indenté lisible
<code>df.to_json("f.json", lines=True)</code>	Format JSONL (une ligne = un objet)
<code>df.to_json("f.json", date_format="iso")</code>	Dates en format ISO 8601
<code>df.to_json("f.json", date_format="epoch")</code>	Dates en timestamp Unix
<code>df.to_json("f.json", date_unit="s")</code>	Unité des timestamps (s/ms/us/ns)
<code>df.to_json("f.json", double_precision=10)</code>	Précision des flottants
<code>df.to_json("f.json", force_ascii=True)</code>	Force l'encodage ASCII
<code>df.to_json("f.json", force_ascii=False)</code>	Conserve les caractères unicode
<code>df.to_json("f.json", default_handler=str)</code>	Gère les types non sérialisables
<code>df.to_json("f.json", compression="gzip")</code>	Compresse le JSON
<code>df.to_json(None)</code>	Retourne une chaîne JSON

— PARQUET —

<code>df.to_parquet("f.parquet")</code>	Exporte en Parquet
<code>df.to_parquet("f.parquet", index=False)</code>	Sans index
<code>df.to_parquet("f.parquet", compression="snappy")</code>	Compression Snappy (défaut)
<code>df.to_parquet("f.parquet", compression="gzip")</code>	Compression GZIP
<code>df.to_parquet("f.parquet", compression="brotli")</code>	Compression Brotli
<code>df.to_parquet("f.parquet", compression="zstd")</code>	Compression Zstandard
<code>df.to_parquet("f.parquet", compression=None)</code>	Sans compression
<code>df.to_parquet("f.parquet", partition_cols=["année","mois"])</code>	Partitionné
<code>df.to_parquet("dossier/", partition_cols=["région"])</code>	En dossier partitionné
<code>df.to_parquet("f.parquet", schema=schema)</code>	Schéma PyArrow explicite

— FEATHER —

<code>df.to_feather("f.feather")</code>	Exporte en Feather
<code>df.to_feather("f.feather", compression="zstd")</code>	Avec compression

— HDF5 —

<code>df.to_hdf("f.h5", key="data")</code>	Exporte en HDF5 (mode fixe)
<code>df.to_hdf("f.h5", key="data", mode="w")</code>	Crée / écrase le fichier
<code>df.to_hdf("f.h5", key="data", mode="a")</code>	Ajoute au fichier
<code>df.to_hdf("f.h5", key="data", format="table")</code>	Format table (requêttable)
<code>df.to_hdf("f.h5", key="data", format="fixed")</code>	Format fixe (rapide)

<code>df.to_hdf("f.h5", key="data", complevel=9)</code>	Niveau de compression (0-9)
<code>df.to_hdf("f.h5", key="data", complib="zlib")</code>	Bibliothèque de compression
<code>df.to_hdf("f.h5", key="data", append=True)</code>	Ajoute des données
<code>df.to_hdf("f.h5", key="data", data_columns=True)</code>	Indexe toutes les colonnes
<code>df.to_hdf("f.h5", key="data", min_itemsize={"col": 50})</code>	Taille min. chaînes

PICKLE

<code>df.to_pickle("f.pkl")</code>	Séréalise en Pickle
<code>df.to_pickle("f.pkl", compression="gzip")</code>	Pickle compressé
<code>df.to_pickle("f.pkl", protocol=4)</code>	Protocole Pickle spécifique

SQL

<code>df.to_sql("table", conn)</code>	Exporte en base SQL
<code>df.to_sql("table", conn, if_exists="fail")</code>	Erreur si la table existe (défaut)
<code>df.to_sql("table", conn, if_exists="replace")</code>	Remplace la table
<code>df.to_sql("table", conn, if_exists="append")</code>	Ajoute les données
<code>df.to_sql("table", conn, index=False)</code>	Sans index
<code>df.to_sql("table", conn, index_label="id")</code>	Nom de la colonne d'index
<code>df.to_sql("table", conn, chunksize=1000)</code>	Insère par lots
<code>df.to_sql("table", conn, dtype={"col": sqlalchemy.Text})</code>	Types SQL explicites
<code>df.to_sql("table", conn, schema="public")</code>	Schéma SQL spécifique
<code>df.to_sql("table", conn, method="multi")</code>	INSERT multi-valeurs (plus rapide)
<code>df.to_sql("table", conn, method=insert_func)</code>	Méthode d'insertion personnalisée

AUTRES FORMATS

<code>df.to_html("f.html")</code>	Exporte en HTML
<code>df.to_html("f.html", index=False)</code>	Sans index
<code>df.to_html("f.html", border=0)</code>	Sans bordure
<code>df.to_html("f.html", classes="tableau")</code>	Classe CSS
<code>df.to_html("f.html", na_rep="-")</code>	Représentation NaN
<code>df.to_html("f.html", float_format="%.2f")</code>	Format des flottants
<code>df.to_html("f.html", bold_rows=True)</code>	Lignes en gras
<code>df.to_html("f.html", escape=True)</code>	Échappe le HTML dans les données
<code>df.to_html("f.html", render_links=True)</code>	Rend les URLs cliquables
<code>df.to_html(None)</code>	Retourne une chaîne HTML
<code>df.to_xml("f.xml")</code>	Exporte en XML
<code>df.to_xml("f.xml", index=False)</code>	Sans index
<code>df.to_xml("f.xml", root_name="données")</code>	Nom de l'élément racine
<code>df.to_xml("f.xml", row_name="ligne")</code>	Nom des éléments lignes
<code>df.to_xml("f.xml", attr_cols=["id"])</code>	Colonnes exportées comme attributs
<code>df.to_xml("f.xml", elem_cols=["nom"])</code>	Colonnes exportées comme éléments
<code>df.to_xml("f.xml", namespaces={"ns": "url"})</code>	Espaces de noms XML
<code>df.to_xml("f.xml", encoding="utf-8")</code>	Encodage
<code>df.to_xml(None)</code>	Retourne une chaîne XML
<code>df.to_markdown()</code>	Retourne une chaîne Markdown
<code>df.to_markdown(index=False)</code>	Sans index
<code>df.to_markdown(tablefmt="grid")</code>	Style de tableau
<code>df.to_latex()</code>	Retourne une chaîne LaTeX
<code>df.to_latex(index=False)</code>	Sans index
<code>df.to_latex(caption="Mon tableau")</code>	Légende
<code>df.to_latex(label="tab:monlabel")</code>	Label de référence
<code>df.to_latex(longtable=True)</code>	Tableau long (multi-pages)
<code>df.to_latex(float_format="%.2f")</code>	Format des flottants
<code>df.to_clipboard()</code>	Copie dans le presse-papiers
<code>df.to_clipboard(index=False)</code>	Sans index
<code>df.to_clipboard(sep="\t")</code>	Séparateur
<code>df.to_string()</code>	Retourne une chaîne lisible
<code>df.to_string(index=False)</code>	Sans index
<code>df.to_string(max_rows=20)</code>	Nombre max de lignes
<code>df.to_string(max_cols=10)</code>	Nombre max de colonnes
<code>df.to_string(float_format="%.3f")</code>	Format des flottants
<code>df.to_string(na_rep="NaN")</code>	Représentation NaN
<code>df.to_dict()</code>	Convertit en dictionnaire Python
<code>df.to_dict("records")</code>	Liste de dicts (une ligne = un dict)
<code>df.to_dict("list")</code>	Dict de listes par colonne
<code>df.to_dict("series")</code>	Dict de Series par colonne
<code>df.to_dict("split")</code>	Dict avec keys/index/data
<code>df.to_dict("index")</code>	Dict de dicts indexé par l'index
<code>df.to_dict("tight")</code>	Dict compact avec toutes infos
<code>df.to_records()</code>	Convertit en tableau numpy recarray
<code>df.to_records(index=True)</code>	Avec index (défaut)
<code>df.to_numpy()</code>	Retourne un array numpy 2D
<code>df.to_numpy(dtype=float)</code>	Array numpy typé
<code>df.to_numpy(na_value=0)</code>	Remplace NaN avant conversion

EXEMPLES GÉNÉRAUX :

```
df.to_csv("résultat.csv", index=False, sep=";", encoding="utf-8")
df.to_parquet("archive.parquet", compression="zstd", index=False)
with pd.ExcelWriter("rapport.xlsx") as writer:
    df1.to_excel(writer, sheet_name="Ventes", index=False)
    df2.to_excel(writer, sheet_name="Clients", index=False)
df.to_json("data.jsonl", orient="records", lines=True)
```

[df] Exploration – info, describe, head, tail, shape...

DESCRIPTION : Inspecte la structure, les types et le contenu d'un DataFrame

POURQUOI : Première étape de toute analyse de données.

CONTEXTE : EDA (Exploratory Data Analysis), diagnostic, audit de données.

FONCTIONS PRINCIPALES :

df.head(n=5)	N premières lignes
df.tail(n=5)	N dernières lignes
df.sample(n=10)	N lignes aléatoires
df.sample(frac=0.1)	Fraction aléatoire (10%)
df.sample(n=10, random_state=42)	Reproductible
df.sample(weights="col")	Échantillonnage pondéré
df.shape	(nb_lignes, nb_colonnes)
df.size	Nombre total de cellules
df.ndim	Nombre de dimensions
df.dtypes	Types de chaque colonne
df.columns	Liste des colonnes
df.index	Index du DataFrame
df.axes	[index, colonnes]
df.values	Données brutes (array numpy)
df.info()	Résumé : types, nulls, mémoire
df.info(memory_usage="deep")	Taille mémoire précise
df.info(verbose=False)	Résumé compact
df.describe()	Statistiques descriptives (num)
df.describe(include="all")	Inclut colonnes catégorielles
df.describe(include="object")	Colonnes texte seulement
df.describe(include=[np.number])	Colonnes numériques seulement
df.describe(percentiles=[.1,.5,.9])	Percentiles personnalisés
df.memory_usage(deep=True)	Mémoire par colonne en octets
df.memory_usage(deep=True).sum()	Mémoire totale
df.count()	Nombre de valeurs non-nulles
df.nunique()	Nombre de valeurs uniques/colonne
df.nunique(axis=1)	Nombre de valeurs uniques/ligne
df["col"].unique()	Valeurs uniques d'une colonne
df["col"].value_counts()	Fréquences des valeurs
df["col"].value_counts(normalize=True)	Fréquences relatives (%)
df["col"].value_counts(ascending=True)	Ordre croissant
df["col"].value_counts(dropna=False)	Inclut les NaN
df["col"].value_counts(bins=5)	Par intervalles (colonnes numériques)
df.empty	True si le DataFrame est vide

EXEMPLES :

```
print(df.shape, df.dtypes)
df.info(memory_usage="deep")
df["catégorie"].value_counts().head(10)
```

[df] Création et construction

DESCRIPTION : Crée des DataFrames et Series depuis diverses sources

POURQUOI : Construire des données de test, transformer des structures existantes.

CONTEXTE : Tests unitaires, données synthétiques, conversion de structures Python.

FONCTIONS PRINCIPALES :

pd.DataFrame({"a": [1,2], "b": [3,4]})	Depuis un dictionnaire
pd.DataFrame([(1,2),(3,4)], columns=["a","b"])	Depuis une liste de tuples
pd.DataFrame(array, columns=["a","b"])	Depuis un array numpy
pd.DataFrame.from_dict(d)	Depuis un dictionnaire
pd.DataFrame.from_dict(d, orient="index")	Dict inversé (index=clés)
pd.DataFrame.from_records(liste)	Depuis une liste de dicts
pd.Series([1, 2, 3])	Crée une Series
pd.Series({"a": 1, "b": 2})	Series depuis un dict
pd.RangeIndex(start=0, stop=10, step=1)	Index numérique
pd.Index(["a", "b", "c"])	Index personnalisé
pd.MultiIndex.from_tuples([("a",1),("b",2)])	Index multi-niveaux
pd.date_range("2024-01-01", periods=365)	Index de dates
pd.date_range("2024-01-01", "2024-12-31", freq="D")	Plage de dates
pd.date_range("2024-01-01", periods=12, freq="MS")	Début de mois

```

pd.date_range("2024-01-01", periods=12, freq="ME")    Fin de mois
pd.timedelta_range("0 days", periods=10)             Index de durées
pd.period_range("2024-01", periods=12, freq="M")      Index de périodes
pd.interval_range(start=0, end=10, periods=5)         Index d'intervalles
pd.bdate_range("2024-01-01", periods=20)             Jours ouvrables

```

EXEMPLES :

```

df = pd.DataFrame({"nom": ["Alice", "Bob"], "age": [30, 25], "ville": ["Paris", "Lyon"]})
dates = pd.date_range("2024-01-01", periods=100, freq="D")
df = pd.DataFrame({"valeur": range(100)}, index=dates)

```

[df] Sélection et indexation – loc, iloc, at, iat

DESCRIPTION : Accède et sélectionne des sous-ensembles du DataFrame
 POURQUOI : Extraire des lignes, colonnes, cellules spécifiques.
 CONTEXTE : Filtrage, extraction, mise à jour ciblée.

FONCTIONS PRINCIPALES :

df["col"]	Sélectionne une colonne (Series)
df[["col1", "col2"]]	Sélectionne plusieurs colonnes
df.loc[label]	Ligne par label d'index
df.loc["a": "c"]	Plage de labels
df.loc[:, "col"]	Toutes lignes, une colonne
df.loc[df["col"] > 5]	Sélection par condition
df.loc[condition, "col"]	Ligne+colonne par condition
df.loc[condition, ["c1", "c2"]]	Lignes + plusieurs colonnes
df.iloc[0]	Première ligne (par position)
df.iloc[0:5]	5 premières lignes
df.iloc[:, 0]	Première colonne
df.iloc[0:5, 1:3]	Sous-matrice par position
df.iloc[-1]	Dernière ligne
df.iloc[:, -1]	Dernière colonne
df.at[label, "col"]	Valeur unique par label (rapide)
df.iat[0, 1]	Valeur unique par position (rapide)
df.get("col", default=None)	Colonne avec valeur par défaut
df.xs("label", level=0)	Coupe transversale (MultiIndex)
df.take([0, 2, 4])	Lignes par position (liste)
pd.IndexSlice[:]	Raccourci pour slicing MultiIndex

EXEMPLES :

```

df.loc[df["age"] > 30, ["nom", "ville"]]
df.iloc[0:100, [0, 2, 5]]
df.at[0, "nom"] = "Charlie"

```

[df] Filtrage et requêtes – query, filter, where, mask, isin

DESCRIPTION : Filtre les lignes et colonnes selon des conditions
 POURQUOI : Sélectionner des sous-ensembles de données pertinents.
 CONTEXTE : Analyse exploratoire, nettoyage, segmentation.

FONCTIONS PRINCIPALES :

df[df["col"] > 10]	Filtre par condition simple
df[(df["a"] > 5) & (df["b"] == "X")]	Conditions multiples (ET)
df[(df["a"] > 5) (df["b"] == "X")]	Conditions multiples (OU)
df[~(df["col"] > 10)]	Négation d'une condition
df.query("col > 10")	Filtre par expression string
df.query("a > @seuil")	Utilise une variable locale
df.query("col in @liste")	Filtre avec @liste
df.query("col.str.startswith('A')")	Méthode str dans query
df.query("index > 5")	Filtre sur l'index
df["col"].isin([1, 2, 3])	Masque : valeurs dans une liste
df[df["col"].isin([1, 2, 3])]	Filtre avec isin
df[~df["col"].isin([1, 2, 3])]	Exclusion avec isin
df.filter(items=["col1", "col2"])	Filtre les colonnes par nom
df.filter(like="prix")	Colonnes contenant "prix"
df.filter(regex="^col_")	Colonnes par regex
df.filter(axis=0, like="Paris")	Filtre les lignes par label
df.where(condition)	Garde les vrais, NaN ailleurs
df.where(condition, other=-1)	Remplace les faux par -1
df.mask(condition)	Remplace les vrais par NaN
df.mask(condition, other=0)	Remplace les vrais par 0
df["col"].between(a, b)	Valeurs entre a et b inclus
df["col"].between(a, b, inclusive="left")	Borne gauche incluse seulement
df.truncate(before=5, after=10)	Tronque par index
df.asof(index)	Dernière valeur connue avant index

EXEMPLES :

```

df.query("age >= 18 and ville == 'Paris'")

```

```
df[df["catégorie"].isin(["A", "B", "C"])]
df.filter(regex="^montant_")
```

[df] Modification et transformation

DESCRIPTION : Modifie la structure ou les valeurs d'un DataFrame
POURQUOI : Préparer, nettoyer et enrichir les données.
CONTEXTE : Feature engineering, nettoyage, transformation de colonnes.
FONCTIONS PRINCIPALES :

<code>df["col"] = valeur</code>	Ajoute ou modifie une colonne
<code>df.assign(col=valeur)</code>	Ajoute une colonne (retourne copie)
<code>df.assign(col=lambda x: x["a"] + x["b"])</code>	Colonne calculée
<code>df.assign(c1=..., c2=...)</code>	Plusieurs colonnes en une fois
<code>df.insert(pos, "col", valeur)</code>	Insère une colonne à la position
<code>df.pop("col")</code>	Supprime et retourne une colonne
<code>df.drop("col", axis=1)</code>	Supprime une colonne
<code>df.drop(["c1", "c2"], axis=1)</code>	Supprime plusieurs colonnes
<code>df.drop(index=0)</code>	Supprime une ligne
<code>df.drop(index=[0,1,2])</code>	Supprime plusieurs lignes
<code>df.drop(labels=["a", "b"], axis=0)</code>	Supprime des lignes par label
<code>df.rename(columns={"ancien": "nouveau"})</code>	Renomme des colonnes
<code>df.rename(columns=str.lower)</code>	Renomme toutes les colonnes (fonction)
<code>df.rename(index={0: "a"})</code>	Renomme des lignes
<code>df.rename_axis("nom_index")</code>	Renomme l'axe d'index
<code>df.set_index("col")</code>	Définit une colonne comme index
<code>df.reset_index()</code>	Réinitialise l'index (en colonne)
<code>df.reset_index(drop=True)</code>	Réinitialise sans garder l'ancien
<code>df.set_axis(["a", "b"], axis=1)</code>	Redéfinit les noms d'axes
<code>df.reindex(nouvel_index)</code>	Réindexe (ajoute NaN si manquant)
<code>df.reindex_like(autre_df)</code>	Réindexe comme un autre DataFrame
<code>df.copy()</code>	Copie complète (deep copy)
<code>df.copy(deep=False)</code>	Copie superficielle (vue)
<code>df.astype({"col": float})</code>	Change le type d'une colonne
<code>df.astype(str)</code>	Tout convertir en string
<code>df.convert_dtypes()</code>	Inférence automatique des types
<code>df.infer_objects()</code>	Inférence des types objets
<code>df.add_prefix("préfixe_")</code>	Ajoute un préfixe aux colonnes
<code>df.add_suffix("_suffixe")</code>	Ajoute un suffixe aux colonnes
<code>df.pipe(fonction)</code>	Applique une fonction au DataFrame
<code>df.eval("c = a + b")</code>	Évalue une expression
<code>df.eval("c = a + b", inplace=True)</code>	Évalue en place
<code>df.update(autre_df)</code>	Met à jour avec un autre DataFrame
<code>df.set_flags(allows_duplicate_labels=False)</code>	Interdit les labels dupliqués

EXEMPLES :

```
df = df.assign(total=df["qté"] * df["prix"], tva=lambda x: x["total"] * 0.2)
df.rename(columns=str.lower).reset_index(drop=True)
```

[df] Nettoyage – dropna, fillna, replace, duplicates

DESCRIPTION : Gère les valeurs manquantes, doublons et données invalides
POURQUOI : Les données réelles sont rarement propres. Étape cruciale avant analyse.
CONTEXTE : Préparation de données, qualité des données, data cleaning.
FONCTIONS PRINCIPALES :

<code>df.isnull()</code>	Masque des valeurs nulles
<code>df.isna()</code>	Identique à isnull()
<code>df.notnull()</code>	Masque des valeurs non-nulles
<code>df.notna()</code>	Identique à notnull()
<code>df.isnull().sum()</code>	Nombre de nuls par colonne
<code>df.isnull().sum().sum()</code>	Nombre total de nuls
<code>df.isnull().mean()</code>	Proportion de nuls par colonne
<code>df.dropna()</code>	Supprime les lignes avec NaN
<code>df.dropna(axis=1)</code>	Supprime les colonnes avec NaN
<code>df.dropna(how="all")</code>	Supprime seulement si tout NaN
<code>df.dropna(how="any")</code>	Supprime si au moins un NaN (défaut)
<code>df.dropna(subset=["col"])</code>	Supprime si NaN dans col spécifique
<code>df.dropna(subset=["c1", "c2"])</code>	Supprime si NaN dans l'une des cols
<code>df.dropna(thresh=N)</code>	Garde si au moins N valeurs non-null
<code>df.fillna(0)</code>	Remplace NaN par 0
<code>df.fillna(method="ffill")</code>	Forward fill (propagation avant)
<code>df.fillna(method="bfill")</code>	Backward fill (propagation arrière)
<code>df.fillna(limit=3)</code>	Limite le nombre de remplissages
<code>df.ffill()</code>	Raccourci forward fill
<code>df.ffill(axis=1)</code>	Forward fill sur les colonnes

df.bfill()	Raccourci backward fill
df.fillna(df.mean())	Remplace par la moyenne
df.fillna(df.median())	Remplace par la médiane
df.fillna(df.mode().iloc[0])	Remplace par le mode
df.fillna({"col1": 0, "col2": "inconnu"})	Valeurs différentes par colonne
df.interpolate()	Interpolation linéaire
df.interpolate(method="time")	Interpolation temporelle
df.interpolate(method="polynomial", order=2)	Interpolation polynomiale
df.interpolate(method="spline", order=3)	Interpolation spline
df.interpolate(limit=2)	Limite l'interpolation
df.interpolate(limit_direction="both")	Dans les deux sens
df.replace(ancien, nouveau)	Remplace une valeur
df.replace([1,2], [10,20])	Remplace plusieurs valeurs
df.replace({"col": {1: 10}})	Remplace dans une colonne
df.replace(regex=True, to_replace=r"\s+", value=" ")	Remplacement par regex
df.replace({np.nan: 0})	Remplace NaN
df.duplicated()	Masque des lignes dupliquées
df.duplicated(subset=["col"])	Dupliqués sur une colonne
df.duplicated(keep="first")	Garde la première occurrence (défaut)
df.duplicated(keep="last")	Garde la dernière occurrence
df.duplicated(keep=False)	Marque toutes les occurrences
df.drop_duplicates()	Supprime les doublons
df.drop_duplicates(subset=["col"])	Déduplique sur une colonne
df.drop_duplicates(keep=False)	Supprime toutes les occurrences
pd.isna(valeur)	Teste si une valeur est NaN
pd.notna(valeur)	Teste si une valeur n'est pas NaN

EXEMPLES :

```
df = df.dropna(subset=["id", "date"]).fillna({"montant": 0, "statut": "inconnu"})
df = df.drop_duplicates(subset=["id"], keep="last")
```

[df] Opérations sur les chaînes – str.*

DESCRIPTION : Applique des opérations de chaîne à une colonne de type texte
POURQUOI : Nettoyage et transformation de données textuelles.
CONTEXTE : Normalisation de données, extraction d'informations, parsing.

FONCTIONS PRINCIPALES :

df["col"].str.lower()	Minuscules
df["col"].str.upper()	Majuscules
df["col"].str.strip()	Supprime espaces début/fin
df["col"].str.lstrip()	Supprime espaces à gauche
df["col"].str.rstrip()	Supprime espaces à droite
df["col"].str.strip("_")	Supprime un caractère spécifique
df["col"].str.replace("a","b")	Remplace une sous-chaîne
df["col"].str.replace(r"\s+", " ", regex=True)	Remplacement par regex
df["col"].str.replace(r"[^a-z]", "", regex=True)	Supprime les non-lettres
df["col"].str.contains("motif")	Test d'inclusion (masque bool)
df["col"].str.contains("motif", na=False)	Gère les NaN
df["col"].str.contains("motif", case=False)	Insensible à la casse
df["col"].str.contains("motif", regex=True)	Recherche regex
df["col"].str.startswith("préfixe")	Commence par
df["col"].str.endswith("suffixe")	Finit par
df["col"].str.match("^regex\$")	Correspond à un regex (depuis début)
df["col"].str.fullmatch("^regex\$")	Correspondance totale
df["col"].str.extract(r"(\d+)")	Extrait le premier groupe regex
df["col"].str.extract(r"(\d+)", expand=True)	Retourne un DataFrame
df["col"].str.extractall(r"(\d+)")	Extrait tous les groupes
df["col"].str.findall(r"\d+")	Trouve toutes les occurrences (liste)
df["col"].str.split(",")	Divise en liste
df["col"].str.split(",", expand=True)	Divise en colonnes
df["col"].str.split(",", n=1)	Divise au maximum N fois
df["col"].str.rsplit(",", n=1)	Divise depuis la droite
df["col"].str.join(sep)	Joint une liste en chaîne
df["col"].str.cat(sep=", ")	Concatène toutes les valeurs
df["col"].str.cat(other_series, sep="-")	Concatène avec une autre Series
df["col"].str.len()	Longueur de chaque chaîne
df["col"].str.count("motif")	Compte les occurrences
df["col"].str.get(i)	i-ème caractère
df["col"].str[0]	Premier caractère (raccourci)
df["col"].str[1:4]	Sous-chaîne [1:4]
df["col"].str.slice(0, 5)	Sous-chaîne [0:5]
df["col"].str.slice_replace(1, 3, "XX")	Remplace une portion
df["col"].str.pad(width=10, fillchar=" ")	Aligne sur N caractères (centre)
df["col"].str.pad(width=10, side="right")	Aligne à gauche
df["col"].str.zfill(5)	Complète avec des zéros

df["col"].str.ljust(10)	Aligne à gauche
df["col"].str.rjust(10)	Aligne à droite
df["col"].str.center(10)	Centre la chaîne
df["col"].str.title()	Première lettre de chaque mot en maj
df["col"].str.capitalize()	Première lettre maj, reste min
df["col"].str.swapcase()	Inverse la casse
df["col"].str.isdigit()	Teste si numérique
df["col"].str.isalpha()	Teste si alphabétique
df["col"].str.isalnum()	Teste si alphanumérique
df["col"].str.isnumeric()	Teste si numérique (unicode)
df["col"].str.isspace()	Teste si que des espaces
df["col"].str.islower()	Teste si tout en minuscules
df["col"].str.isupper()	Teste si tout en majuscules
df["col"].str.istitle()	Teste si titre
df["col"].str.encode("utf-8")	Encode en bytes
df["col"].str.decode("utf-8")	Décode des bytes
df["col"].str.normalize("NFC")	Normalisation unicode
df["col"].str.translate(table)	Traduction de caractères
df["col"].str.wrap(width=80)	Retour à la ligne automatique
df["col"].str.removeprefix("préfixe")	Supprime un préfixe (Python 3.9+)
df["col"].str.removesuffix("suffixe")	Supprime un suffixe (Python 3.9+)
df["col"].str.repeat(3)	Répète la chaîne N fois
df["col"].str.get_dummies(sep=" ")	One-hot encoding sur séparateur

EXEMPLES :

```
df["nom"] = df["nom"].str.strip().str.lower()
df[df["email"].str.contains(r"@w+\.w+", regex=True, na=False)]
df[["prénom", "nom"]] = df["nom_complet"].str.split(" ", n=1, expand=True)
```

[df] Dates et temps – dt.* / to_datetime / date_range

DESCRIPTION : Manipule les données temporelles et les séries chronologiques

POURQUOI : Les données temporelles nécessitent un traitement spécifique.

CONTEXTE : Séries temporelles, analyses par période, resampling.

FONCTIONS PRINCIPALES :

pd.to_datetime(df["col"])	Convertit en datetime
pd.to_datetime(df["col"], format="%d/%m/%Y")	Format personnalisé
pd.to_datetime(df["col"], errors="coerce")	NaT si invalide
pd.to_datetime(df["col"], errors="ignore")	Laisse la valeur d'origine si erreur
pd.to_datetime(df["col"], utc=True)	Force UTC
pd.to_datetime(df["col"], dayfirst=True)	Format JJ/MM/AAAA
pd.to_datetime(df["col"], unit="s")	Timestamp Unix en secondes
pd.to_datetime(df["col"], unit="ms")	Timestamp Unix en ms
pd.to_datetime({"year":..., "month":..., "day":...})	Depuis colonnes séparées
pd.to_timedelta(df["col"])	Convertit en timedelta
pd.to_timedelta(df["col"], unit="s")	Depuis secondes
pd.Timestamp("2024-01-01")	Crée un timestamp
pd.Timestamp("now")	Timestamp actuel
pd.NaT	Valeur manquante temporelle
df["col"].dt.year	Année
df["col"].dt.month	Mois (1-12)
df["col"].dt.day	Jour (1-31)
df["col"].dt.hour	Heure (0-23)
df["col"].dt.minute	Minute (0-59)
df["col"].dt.second	Seconde (0-59)
df["col"].dt.microsecond	Microseconde
df["col"].dt.nanosecond	Nanoseconde
df["col"].dt.dayofweek	Jour de la semaine (0=lundi)
df["col"].dt.day_of_week	Identique (alias)
df["col"].dt.day_name()	Nom du jour ("Monday"...)
df["col"].dt.day_name(locale="fr_FR")	Nom du jour en français
df["col"].dt.month_name()	Nom du mois ("January"...)
df["col"].dt.month_name(locale="fr_FR")	Nom du mois en français
df["col"].dt.dayofyear	Jour de l'année (1-365/366)
df["col"].dt.day_of_year	Identique (alias)
df["col"].dt.weekofyear	Semaine ISO de l'année
df["col"].dt.isocalendar()	Année, semaine et jour ISO
df["col"].dt.quarter	Trimestre (1-4)
df["col"].dt.is_month_start	Vrai si début de mois
df["col"].dt.is_month_end	Vrai si fin de mois
df["col"].dt.is_quarter_start	Vrai si début de trimestre
df["col"].dt.is_quarter_end	Vrai si fin de trimestre
df["col"].dt.is_year_start	Vrai si début d'année
df["col"].dt.is_year_end	Vrai si fin d'année
df["col"].dt.is_leap_year	Vrai si année bissextile

<code>df["col"].dt.days_in_month</code>	Nombre de jours dans le mois
<code>df["col"].dt.date</code>	Date seule (sans heure)
<code>df["col"].dt.time</code>	Heure seule (sans date)
<code>df["col"].dt.timez</code>	Heure avec timezone
<code>df["col"].dt.normalize()</code>	Réinitialise l'heure à 00:00
<code>df["col"].dt.floor("h")</code>	Arrondi à l'heure inférieure
<code>df["col"].dt.ceil("h")</code>	Arrondi à l'heure supérieure
<code>df["col"].dt.round("h")</code>	Arrondi à l'heure la plus proche
<code>df["col"].dt.tz_localize("UTC")</code>	Applique un fuseau horaire
<code>df["col"].dt.tz_localize(None)</code>	Supprime le fuseau horaire
<code>df["col"].dt.tz_convert("Europe/Paris")</code>	Convertit le fuseau horaire
<code>df["col"].dt.tz</code>	Fuseau horaire de la colonne
<code>df["col"].dt.strftime("%d/%m/%Y")</code>	Formate en chaîne
<code>df["col"].dt.strftime("%Y-%m-%d %H:%M")</code>	Format avec heure
<code>df["col"].dt.total_seconds()</code>	Durée totale en secondes (timedelta)
<code>df["col"].dt.days</code>	Jours entiers (timedelta)
<code>df["col"].dt.seconds</code>	Secondes restantes (timedelta)
<code>df["col"].dt.microseconds</code>	Microsecondes restantes
<code>df["col"].dt.components</code>	Décompose un timedelta en colonnes
<code>df["col"].dt.to_period("M")</code>	Convertit en période mensuelle
<code>df["col"].dt.to_period("Q")</code>	Convertit en période trimestrielle
<code>df["col"].dt.to_pydatetime()</code>	Convertit en datetime Python
<code>df["col"].dt.to_pytimedelta()</code>	Convertit en timedelta Python
<code>df.resample("ME").sum()</code>	Agrège par fin de mois
<code>df.resample("W").mean()</code>	Agrège par semaine
<code>df.resample("QE").agg({"col": "sum"})</code>	Agrège par trimestre
<code>df.resample("D").interpolate()</code>	Resampling + interpolation
<code>df.resample("h").ffill()</code>	Resampling + forward fill
<code>df.asfreq("D")</code>	Change la fréquence (sans agréger)
<code>df.asfreq("D", method="ffill")</code>	Change freq avec remplissage
<code>df.at_time("09:00")</code>	Filtre par heure exacte
<code>df.between_time("09:00", "17:00")</code>	Filtre par plage horaire
<code>df.shift(1)</code>	Décale d'une période
<code>df.shift(1, freq="D")</code>	Décale d'un jour
<code>df.diff()</code>	Différence avec la période précédente
<code>df.pct_change()</code>	Variation en pourcentage
<code>pd.offsets.Day(1)</code>	Offset d'un jour
<code>pd.offsets.MonthEnd()</code>	Offset fin de mois
<code>pd.offsets.BusinessDay()</code>	Offset jour ouvrable
<code>pd.offsets.YearEnd()</code>	Offset fin d'année
<code>pd.offsets.Hour(3)</code>	Offset de 3 heures

EXEMPLES :

```
df["date"] = pd.to_datetime(df["date"], format="%d/%m/%Y")
df["année"] = df["date"].dt.year
df.set_index("date").resample("ME").agg({"ventes": "sum", "clients": "nunique"})
df["date"] = pd.to_datetime(df[["year", "month", "day"]])
```

[df] Statistiques et mathématiques

DESCRIPTION : Calcule des statistiques descriptives et des opérations mathématiques
 POURQUOI : Analyser et résumer les données numériques.
 CONTEXTE : Analyse exploratoire, rapports statistiques, feature engineering.

FONCTIONS PRINCIPALES :

<code>df.sum()</code>	Somme par colonne
<code>df.sum(axis=1)</code>	Somme par ligne
<code>df.sum(skipna=True)</code>	Ignore les NaN (défaut)
<code>df.mean()</code>	Moyenne
<code>df.mean(numeric_only=True)</code>	Colonnes numériques seulement
<code>df.median()</code>	Médiane
<code>df.mode()</code>	Mode (valeur la plus fréquente)
<code>df.min()</code>	Minimum
<code>df.max()</code>	Maximum
<code>df.std()</code>	Écart-type
<code>df.std(ddof=1)</code>	Écart-type échantillon (défaut)
<code>df.std(ddof=0)</code>	Écart-type population
<code>df.var()</code>	Variance
<code>df.sem()</code>	Erreur standard de la moyenne
<code>df.skew()</code>	Asymétrie (skewness)
<code>df.kurt()</code>	Kurtosis
<code>df.abs()</code>	Valeur absolue
<code>df.round(2)</code>	Arrondi à N décimales
<code>df.round({"a": 2, "b": 0})</code>	Arrondi différent par colonne
<code>df.clip(lower=0, upper=100)</code>	Borne les valeurs
<code>df.clip(lower=df["col"].quantile(0.01))</code>	Borne par quantile

<code>df.quantile(0.25)</code>	1er quartile
<code>df.quantile([0.25, 0.5, 0.75])</code>	Quartiles multiples
<code>df.quantile(q=0.95, interpolation="linear")</code>	Interpolation du quantile
<code>df.cumsum()</code>	Somme cumulée
<code>df.cumprod()</code>	Produit cumulé
<code>df.cummax()</code>	Maximum cumulé
<code>df.cummin()</code>	Minimum cumulé
<code>df.diff(periods=1)</code>	Différence avec la ligne N-1
<code>df.diff(periods=-1)</code>	Différence avec la ligne suivante
<code>df.pct_change()</code>	Variation en %
<code>df.pct_change(periods=12)</code>	Variation sur 12 périodes
<code>df.rank()</code>	Rang de chaque valeur
<code>df.rank(method="dense")</code>	Rang dense (sans saut)
<code>df.rank(method="min")</code>	Rang minimum (ex-aequo)
<code>df.rank(method="average")</code>	Rang moyen (ex-aequo, défaut)
<code>df.rank(pct=True)</code>	Rang en percentile
<code>df.rank(ascending=False)</code>	Rang décroissant
<code>df.corr()</code>	Matrice de corrélation (Pearson)
<code>df.corr(method="spearman")</code>	Corrélation de Spearman
<code>df.corr(method="kendall")</code>	Corrélation de Kendall
<code>df.cov()</code>	Matrice de covariance
<code>df["a"].corr(df["b"])</code>	Corrélation entre deux Series
<code>df["a"].cov(df["b"])</code>	Covariance entre deux Series
<code>df.dot(autre)</code>	Produit matriciel
<code>df.prod()</code>	Produit de toutes les valeurs
<code>df.idxmax()</code>	Index de la valeur max
<code>df.idxmin()</code>	Index de la valeur min
<code>df.nlargest(n, "col")</code>	N plus grandes valeurs
<code>df.nsmallest(n, "col")</code>	N plus petites valeurs
<code>df.count()</code>	Nombre de valeurs non-nulles
<code>df.eval("c = a + b", inplace=True)</code>	Calcul d'expression
<code>df.add(autre, fill_value=0)</code>	Addition avec fill
<code>df.sub(autre, fill_value=0)</code>	Soustraction avec fill
<code>df.mul(autre, fill_value=1)</code>	Multiplication avec fill
<code>df.div(autre, fill_value=1)</code>	Division avec fill
<code>df.pow(2)</code>	Élévation à la puissance
<code>df.mod(autre)</code>	Modulo
<code>df.floordiv(autre)</code>	Division entière
<code>df.truediv(autre)</code>	Division réelle
<code>df.all()</code>	Vrai si toutes valeurs vraies
<code>df.any()</code>	Vrai si au moins une valeur vraie

EXEMPLES :

```
df[["a", "b"]].corr()
df.nlargest(10, "montant")[["client", "montant"]]
```

[df] Tri – sort_values, sort_index

DESCRIPTION : Trie les lignes ou colonnes d'un DataFrame
 POURQUOI : Ordonner les données pour l'affichage, les analyses ou les jointures.
 CONTEXTE : Rapports classés, préparation de jointures, classements.

FONCTIONS PRINCIPALES :

<code>df.sort_values("col")</code>	Tri croissant par colonne
<code>df.sort_values("col", ascending=False)</code>	Tri décroissant
<code>df.sort_values(["col1", "col2"])</code>	Tri multi-colonnes
<code>df.sort_values(["col1", "col2"], ascending=[True, False])</code>	Ordre mixte
<code>df.sort_values("col", na_position="first")</code>	NaN en premier
<code>df.sort_values("col", na_position="last")</code>	NaN en dernier (défaut)
<code>df.sort_values("col", key=lambda x: x.str.lower())</code>	Tri avec transformation
<code>df.sort_values("col", inplace=True)</code>	Tri en place
<code>df.sort_index()</code>	Tri par index croissant
<code>df.sort_index(ascending=False)</code>	Tri par index décroissant
<code>df.sort_index(axis=1)</code>	Tri des colonnes par nom
<code>df.sort_index(level=0)</code>	Tri sur un niveau de MultiIndex
<code>df.sort_index(sort_remaining=False)</code>	Ne trie pas les niveaux restants
<code>df.nlargest(n, "col")</code>	N plus grandes valeurs (trié)
<code>df.nsmallest(n, "col")</code>	N plus petites valeurs (trié)
<code>df["col"].argsort()</code>	Indices qui trieraient la colonne

EXEMPLES :

```
df.sort_values(["région", "ca"], ascending=[True, False])
df.nlargest(5, "chiffre_affaires")
```

[df] Regroupement et agrégation – groupby

DESCRIPTION : Regroupe les lignes par valeurs et applique des fonctions d'agrégation
 POURQUOI : Calculer des statistiques par groupe (par région, par catégorie...)
 CONTEXTE : Tableaux de synthèse, rapports d'agrégation, analyses par segment.

FONCTIONS PRINCIPALES :

<code>df.groupby("col")</code>	Groupe par une colonne
<code>df.groupby(["col1", "col2"])</code>	Groupe par plusieurs colonnes
<code>df.groupby("col", as_index=False)</code>	Résultat sans index groupé
<code>df.groupby("col", dropna=False)</code>	Inclut les groupes NaN
<code>df.groupby("col", sort=False)</code>	Ne trie pas les groupes
<code>df.groupby("col", observed=True)</code>	Seulement les catégories observées
<code>df.groupby("col").sum()</code>	Somme par groupe
<code>df.groupby("col").mean()</code>	Moyenne par groupe
<code>df.groupby("col").median()</code>	Médiane par groupe
<code>df.groupby("col").min()</code>	Minimum par groupe
<code>df.groupby("col").max()</code>	Maximum par groupe
<code>df.groupby("col").count()</code>	Comptage (hors NaN) par groupe
<code>df.groupby("col").std()</code>	Écart-type par groupe
<code>df.groupby("col").var()</code>	Variance par groupe
<code>df.groupby("col").first()</code>	Première valeur par groupe
<code>df.groupby("col").last()</code>	Dernière valeur par groupe
<code>df.groupby("col").nunique()</code>	Valeurs uniques par groupe
<code>df.groupby("col").size()</code>	Taille de chaque groupe (avec NaN)
<code>df.groupby("col").agg("sum")</code>	Agrégation simple
<code>df.groupby("col").agg(["sum", "mean"])</code>	Plusieurs agrégations
<code>df.groupby("col").agg({"a": "sum", "b": "mean"})</code>	Agrégation différente par col
<code>df.groupby("col").agg(total=("val", "sum"))</code>	Avec renommage (NamedAgg)
<code>df.groupby("col").agg(</code> <code>total=("montant", "sum"),</code> <code>moyenne=("montant", "mean"),</code> <code>nb=("id", "nunique"))</code>	Exemple complet avec NamedAgg
<code>df.groupby("col").apply(fonction)</code>	Fonction personnalisée
<code>df.groupby("col").transform("mean")</code>	Transforme (même shape que df)
<code>df.groupby("col").transform(lambda x: x - x.mean())</code>	Transform lambda
<code>df.groupby("col").filter(lambda x: len(x)>5)</code>	Filtre des groupes entiers
<code>df.groupby("col").cumsum()</code>	Somme cumulée par groupe
<code>df.groupby("col").rank()</code>	Rang au sein du groupe
<code>df.groupby("col").diff()</code>	Différence au sein du groupe
<code>df.groupby("col").pct_change()</code>	Variation % au sein du groupe
<code>df.groupby("col").head(3)</code>	3 premières lignes par groupe
<code>df.groupby("col").tail(3)</code>	3 dernières lignes par groupe
<code>df.groupby("col").nth(0)</code>	N-ième ligne par groupe
<code>df.groupby("col").sample(n=2)</code>	Échantillon aléatoire par groupe
<code>df.groupby("col").ngroups</code>	Nombre de groupes
<code>df.groupby("col").groups</code>	Dictionnaire des groupes
<code>df.groupby("col").get_group("valeur")</code>	Obtient un groupe spécifique
<code>df.groupby("col").describe()</code>	Statistiques par groupe
<code>df.groupby("col").value_counts()</code>	Comptage par groupe
<code>df.groupby("col").rolling(3).mean()</code>	Rolling au sein du groupe
<code>df.groupby("col").expanding().sum()</code>	Expanding au sein du groupe
<code>df.groupby("col").resample("M").sum()</code>	Resample au sein du groupe
<code>pd.NamedAgg(column="val", aggfunc="sum")</code>	Agrégation nommée
<code>pd.Grouper(freq="ME")</code>	Grouper temporel mensuel
<code>pd.Grouper(key="date", freq="W")</code>	Grouper sur une colonne date

EXEMPLES :

```
df.groupby("région", as_index=False).agg(
    ca_total=("montant", "sum"),
    nb_clients=("client_id", "nunique"),
    panier_moyen=("montant", "mean")
)
```

[df] Fusion et jointure – merge, join, concat

DESCRIPTION : Combine plusieurs DataFrames entre eux
 POURQUOI : Enrichir les données, recouper des tables, assembler des datasets.
 CONTEXTE : Jointures de tables, assemblage de datasets, ETL.

FONCTIONS PRINCIPALES :

<code>pd.merge(df1, df2, on="col")</code>	Jointure sur une colonne commune
<code>pd.merge(df1, df2, on=["c1", "c2"])</code>	Jointure multi-colonnes
<code>pd.merge(df1, df2, how="inner")</code>	Jointure interne (défaut)
<code>pd.merge(df1, df2, how="left")</code>	Jointure gauche
<code>pd.merge(df1, df2, how="right")</code>	Jointure droite
<code>pd.merge(df1, df2, how="outer")</code>	Jointure externe complète
<code>pd.merge(df1, df2, how="cross")</code>	Produit cartésien
<code>pd.merge(df1, df2, left_on="a", right_on="b")</code>	Colonnes de jointure différentes

```

pd.merge(df1, df2, left_index=True, right_on="col") Index gauche + col droite
pd.merge(df1, df2, left_on="col", right_index=True) Col gauche + index droit
pd.merge(df1, df2, left_index=True, right_index=True) Sur les index
pd.merge(df1, df2, suffixes=("_g", "_d")) Suffixes en cas de doublon
pd.merge(df1, df2, indicator=True) Colonne _merge pour diagnostic
pd.merge(df1, df2, validate="one_to_one") Valide le type de relation
pd.merge(df1, df2, validate="one_to_many") Valide relation 1-N
pd.merge(df1, df2, validate="many_to_one") Valide relation N-1
pd.merge_asof(df1, df2, on="date") Jointure approximative (timeseries)
pd.merge_asof(df1, df2, on="date", direction="backward") Valeur précédente
pd.merge_asof(df1, df2, on="date", direction="forward") Valeur suivante
pd.merge_asof(df1, df2, on="date", direction="nearest") Valeur la plus proche
pd.merge_asof(df1, df2, on="date", tolerance=pd.Timedelta("1D")) Tolérance
pd.merge_ordered(df1, df2, on="date") Jointure avec tri
pd.merge_ordered(df1, df2, on="date", fill_method="ffill") Avec remplissage
df1.join(df2, how="left") Jointure sur les index
df1.join(df2, on="col") Jointure df1.col = df2.index
df1.join([df2, df3]) Jointure multiple
df1.join(df2, lsuffix="_g", rsuffix="_d") Suffixes
pd.concat([df1, df2]) Empile verticalement (axis=0)
pd.concat([df1, df2], axis=1) Empile horizontalement
pd.concat([df1, df2], ignore_index=True) Réinitialise l'index
pd.concat([df1, df2], keys=["a", "b"]) Ajoute un niveau d'index
pd.concat([df1, df2], join="inner") Garde seulement les colonnes communes
pd.concat([df1, df2], join="outer") Garde toutes les colonnes (défaut)
pd.concat([df1, df2], verify_integrity=True) Vérifie les index dupliqués
pd.concat([df1, df2], sort=True) Trie les colonnes
df1.combine(df2, func) Combine avec une fonction
df1.combine_first(df2) df1 prioritaire, df2 pour combler
df1.update(df2) Met à jour df1 avec df2 (in-place)
df1.align(df2) Aligné deux DataFrames sur les index
df1.align(df2, join="inner") Aligné avec jointure interne

```

EXEMPLES :

```

df = pd.merge(clients, commandes, on="client_id", how="left")
df = pd.concat([df_2022, df_2023, df_2024], ignore_index=True)
df = pd.merge_asof(prix, événements, on="date", direction="backward")

```

[df] Pivotement et restructuration

DESCRIPTION : Restructure la forme du DataFrame (long/large, pivot, melt)

POURQUOI : Passer d'un format long à large ou inversement selon les besoins.

CONTEXTE : Préparation pour visualisation, tableaux croisés, reshaping.

FONCTIONS PRINCIPALES :

```

df.pivot(index="i", columns="c", values="v") Pivoté sans agrégation
df.pivot(index="i", columns="c") Pivoté toutes les colonnes
df.pivot_table(index="i", columns="c", values="v", aggfunc="sum") Pivoté avec agrégation

df.pivot_table(..., fill_value=0) Remplace NaN par 0
df.pivot_table(..., margins=True) Ajoute les totaux (ligne/colonne)
df.pivot_table(..., margins_name="Total") Nom des totaux
df.pivot_table(..., aggfunc=["sum", "mean"]) Plusieurs fonctions d'agrégation
pd.pivot_table(df, values="v", index="i", columns="c", aggfunc="mean") Fonction top-level

df.melt(id_vars=["id"]) Long format (toutes colonnes)
df.melt(id_vars=["id"], value_vars=["a", "b"]) Long format (colonnes choisies)
df.melt(var_name="variable", value_name="val") Noms personnalisés
df.melt(id_vars=["id"], ignore_index=False) Conserve l'index d'origine
pd.melt(df, id_vars=["id"]) Fonction top-level
pd.wide_to_long(df, stubnames=["val"], i="id", j="année") Large → long avec suffixes
pd.wide_to_long(df, stubnames=["val"], i="id", j="année", sep="_") Avec séparateur

df.stack() Colonnes → index (long)
df.stack(level=-1) Dernier niveau de colonnes
df.stack(future_stack=True) Nouveau comportement (pandas 2.x)
df.unstack() Index → colonnes (large)
df.unstack(level=0) Premier niveau d'index
df.unstack(fill_value=0) Remplace NaN
pd.crosstab(df["a"], df["b"]) Tableau croisé de fréquences
pd.crosstab(df["a"], df["b"], normalize=True) En proportions
pd.crosstab(df["a"], df["b"], normalize="index") Normalise par ligne
pd.crosstab(df["a"], df["b"], values=df["v"], aggfunc="sum") Tableau croisé avec valeurs
pd.crosstab(df["a"], df["b"], margins=True) Avec totaux

```

df.explode("col")	Explose une liste en lignes
df.explode(["c1", "c2"])	Explose plusieurs colonnes
df.explode("col", ignore_index=True)	Réinitialise l'index
df.squeeze()	Réduit les dimensions inutiles
df.transpose()	Transpose lignes/colonnes
df.T	Raccourci transpose

EXEMPLES :

```

tableau = df.pivot_table(index="région", columns="catégorie",
                        values="montant", aggfunc="sum", fill_value=0,
                        margins=True, margins_name="TOTAL")

df_long = df.melt(id_vars=["date", "région"], var_name="produit", value_name="ventes")

```

[df] Fenêtres glissantes – rolling, expanding, ewm

DESCRIPTION : Calcule des statistiques sur des fenêtres glissantes ou cumulées
 POURQUOI : Lissage de séries temporelles, moyennes mobiles, tendances.
 CONTEXTE : Finance, séries temporelles, détection de tendances.

FONCTIONS PRINCIPALES :

df.rolling(N)	Fenêtre glissante de N périodes
df.rolling(N).mean()	Moyenne mobile
df.rolling(N).sum()	Somme glissante
df.rolling(N).std()	Écart-type glissant
df.rolling(N).var()	Variance glissante
df.rolling(N).min()	Minimum glissant
df.rolling(N).max()	Maximum glissant
df.rolling(N).median()	Médiane glissante
df.rolling(N).quantile(0.25)	Quantile glissant
df.rolling(N).corr(autre)	Corrélation glissante
df.rolling(N).cov(autre)	Covariance glissante
df.rolling(N).apply(func)	Fonction personnalisée glissante
df.rolling(N).apply(func, raw=True)	Passe un array numpy (plus rapide)
df.rolling(N).agg(["mean", "std"])	Plusieurs agrégations
df.rolling(N, min_periods=1)	Périodes minimum requises
df.rolling(N, center=True)	Fenêtre centrée
df.rolling(N, win_type="gaussian")	Fenêtre gaussienne
df.rolling(N, win_type="hamming")	Fenêtre de Hamming
df.rolling("7D")	Fenêtre glissante temporelle
df.rolling("30D", min_periods=1)	30 jours, dès 1 valeur disponible
df.expanding()	Fenêtre croissante depuis début
df.expanding().mean()	Moyenne cumulative
df.expanding().sum()	Somme cumulative
df.expanding().std()	Écart-type cumulé
df.expanding().min()	Minimum cumulé
df.expanding().max()	Maximum cumulé
df.expanding(min_periods=10)	Minimum de périodes
df.ewm(span=N)	Lissage exponentiel (par span)
df.ewm(span=N).mean()	Moyenne mobile exponentielle
df.ewm(span=N).std()	Écart-type exponentiel
df.ewm(span=N).var()	Variance exponentielle
df.ewm(span=N).corr()	Corrélation exponentielle
df.ewm(alpha=0.3)	Lissage avec alpha ($0 < \alpha \leq 1$)
df.ewm(halflife="7D")	Demi-vie temporelle
df.ewm(halflife=7, times=df["date"])	Demi-vie en unités de temps réelles
df.ewm(com=N)	Centre de masse
df.ewm(adjust=True)	Avec biais initial (défaut)
df.ewm(adjust=False)	Sans correction de biais
df.ewm(ignore_na=True)	Ignore les NaN dans le calcul

EXEMPLES :

```

df["ma_7j"] = df["close"].rolling(7).mean()
df["ema_30"] = df["close"].ewm(span=30).mean()
df["vol_30j"] = df["rendement"].rolling(30).std() * (252**0.5)

```

[df] Application de fonctions – apply, map, transform, pipe

DESCRIPTION : Applique des fonctions personnalisées sur les données
 POURQUOI : Transformer les données avec une logique métier personnalisée.
 CONTEXTE : Feature engineering, transformations complexes, normalisation.

FONCTIONS PRINCIPALES :

df.apply(func)	Applique func sur chaque colonne
df.apply(func, axis=1)	Applique func sur chaque ligne
df.apply(func, result_type="expand")	Résultat expansé en colonnes
df.apply(func, result_type="reduce")	Résultat réduit
df.apply(func, result_type="broadcast")	Même shape que l'entrée

df.apply(func, args=(arg1,))	Arguments positionnels
df.apply(func, kwarg=val)	Arguments nommés
df["col"].apply(func)	Applique func sur une Series
df["col"].apply(func, convert_dtype=True)	Infère le type de retour
df["col"].map(func)	Mappe element-par-element
df["col"].map({"a": 1, "b": 2})	Mappe via un dictionnaire
df["col"].map({"a": 1}, na_action="ignore")	Ignore les NaN
df.map(func)	Applique func sur chaque cellule
df.map(func, na_action="ignore")	Ignore les NaN
df.applymap(func)	Identique à map (déprécié)
df.transform(func)	Transforme, retourne même shape
df.transform(["mean", "std"])	Plusieurs transformations
df.transform({"col1": "sum", "col2": "mean"})	Différentes transfo par colonne
df.groupby("col").transform("mean")	Transforme par groupe
df.groupby("col").transform(lambda x: (x - x.mean()) / x.std())	
df.agg(func)	Agrège sur l'axe
df.agg(["sum", "mean"])	Plusieurs agrégations
df.agg({"col1": "sum", "col2": "mean"})	Agrégation par colonne
df.pipe(func)	Applique func au DataFrame entier
df.pipe(func, arg=val)	Avec arguments supplémentaires
df.pipe(func1).pipe(func2).pipe(func3)	Chainage de fonctions
pd.eval("df['c'] = df['a'] + df['b']")	Évalue une expression globale

EXEMPLES :

```
df["score_norm"] = df.groupby("groupe")["score"].transform(
    lambda x: (x - x.mean()) / x.std())
df.apply(lambda row: row["a"] * row["b"] if row["flag"] else 0, axis=1)
df.pipe(nettoyer).pipe(enrichir).pipe(normaliser)
```

[df] Catégories – astype("category"), cat.*

DESCRIPTION : Gère les données catégorielles pour optimiser mémoire et performance

POURQUOI : Les colonnes catégorielles occupent moins de mémoire et permettent un tri ordonné et des opérations groupby plus rapides.

CONTEXTE : Variables qualitatives, encodage, optimisation mémoire.

FONCTIONS PRINCIPALES :

df["col"].astype("category")	Convertit en catégorielle
pd.Categorical(["a", "b", "a"])	Crée une Series catégorielle
pd.CategoricalDtype(categories=["a", "b"], ordered=True)	Dtype ordonné
df["col"].cat.categories	Liste des catégories
df["col"].cat.codes	Codes numériques des catégories
df["col"].cat.ordered	Vrai si ordonné
df["col"].cat.add_categories(["c"])	Ajoute des catégories
df["col"].cat.remove_categories(["a"])	Supprime des catégories
df["col"].cat.remove_unused_categories()	Supprime les catégories vides
df["col"].cat.set_categories(["a", "b"])	Redéfinit les catégories
df["col"].cat.rename_categories({"a": "A"})	Renomme les catégories
df["col"].cat.reorder_categories(liste)	Réordonne les catégories
df["col"].cat.as_ordered()	Rend la colonne ordonnée
df["col"].cat.as_unordered()	Rend la colonne non-ordonnée
pd.get_dummies(df["col"])	One-hot encoding
pd.get_dummies(df, columns=["c1", "c2"])	One-hot sur colonnes spécifiques
pd.get_dummies(df, drop_first=True)	One-hot sans première catégorie
pd.get_dummies(df, dtype=int)	Résultat en entiers (0/1)
pd.from_dummies(df_dummies)	Inverse du one-hot encoding
pd.factorize(df["col"])	Encodage numérique + index
pd.factorize(df["col"], sort=True)	Encodage trié
pd.factorize(df["col"], na_sentinel=-1)	Sentinel pour NaN
pd.cut(df["col"], bins=5)	Discretisation en intervalles égaux
pd.cut(df["col"], bins=[0, 10, 50, 100])	Discretisation avec bornes
pd.cut(df["col"], bins=5, labels=["TBas", "Bas", "Moy", "Haut", "THaut"])	
pd.cut(df["col"], bins=5, right=False)	Intervalles ouverts à droite
pd.cut(df["col"], bins=5, include_lowest=True)	Inclut la valeur minimale
pd.cut(df["col"], bins=5, retbins=True)	Retourne aussi les bornes
pd.qcut(df["col"], q=4)	Discretisation par quantiles
pd.qcut(df["col"], q=4, labels=["Q1", "Q2", "Q3", "Q4"])	
pd.qcut(df["col"], q=4, duplicates="drop")	Supprime les bornes dupliquées

EXEMPLES :

```
df["taille"] = pd.Categorical(df["taille"], categories=["S", "M", "L", "XL"], ordered=True)
dummies = pd.get_dummies(df[["sexe", "région"]], drop_first=True, dtype=int)
df["tranche_age"] = pd.cut(df["age"], bins=[0, 18, 35, 60, 100],
    labels=["mineur", "jeune", "adulte", "senior"])
```

DESCRIPTION : Manipule les index simples et hiérarchiques
 POURQUOI : L'index est la clé d'accès aux données et des jointures efficaces.
 CONTEXTE : Données hiérarchiques, séries temporelles, jointures par index.

FONCTIONS PRINCIPALES :

df.set_index("col")	Définit une colonne comme index
df.set_index(["col1", "col2"])	Index MultiIndex
df.set_index("col", drop=False)	Garde la colonne dans le df
df.set_index("col", append=True)	Ajoute au MultiIndex existant
df.reset_index()	Remet l'index en colonne
df.reset_index(drop=True)	Supprime l'index
df.reset_index(level=0)	Remet seulement le niveau 0
df.index.rename("nom")	Renomme l'index
df.index.rename(["n1", "n2"])	Renomme un MultiIndex
df.reindex(nouvel_index)	Réindexe (NaN si manquant)
df.reindex(nouvel_index, method="ffill")	Remplit les valeurs manquantes
df.reindex(nouvel_index, fill_value=0)	Remplit avec une valeur fixe
df.reindex(colonnes, axis=1)	Réindexe les colonnes
df.sort_index()	Trie par index
df.sort_index(ascending=False)	Trie par index décroissant
df.sort_index(level=0)	Trie par niveau d'index
df.index.is_unique	Vrai si l'index est unique
df.index.is_monotonic_increasing	Vrai si index croissant
df.index.is_monotonic_decreasing	Vrai si index décroissant
df.index.duplicated()	Masque des index dupliqués
df.index.get_loc("label")	Position d'un label dans l'index
df.index.get_level_values(0)	Valeurs d'un niveau de MultiIndex
pd.MultiIndex.from_tuples(liste)	MultiIndex depuis tuples
pd.MultiIndex.from_product([L1, L2])	MultiIndex produit cartésien
pd.MultiIndex.from_frame(df)	MultiIndex depuis DataFrame
pd.MultiIndex.from_arrays([a1, a2])	MultiIndex depuis arrays
df.xs("val", level=0)	Coupe transversale niveau 0
df.xs(["val1", "val2"])	Coupe transversale MultiIndex
df.swaplevel(0, 1)	Échange deux niveaux d'index
df.droplevel(0)	Supprime un niveau d'index
df.unstack(level=0)	Transforme niveau index en colonnes
df.unstack(level="nom")	Par nom de niveau
df.stack()	Transforme colonnes en index
pd.IndexSlice	Objet pour slicing MultiIndex
df.loc[pd.IndexSlice[:, "val"], :]	Slice sur niveau 1

EXEMPLES :

```
df = df.set_index(["année", "mois"]).sort_index()
df.xs(2024, level="année")
idx = pd.IndexSlice
df.loc[idx[2024, "Jan":"Mar"], :]
```

DESCRIPTION : Fonctions pandas accessibles directement via pd.*
 POURQUOI : Opérations générales non liées à un DataFrame spécifique.
 CONTEXTE : Manipulation de données, conversions, utilitaires.

FONCTIONS PRINCIPALES :

pd.to_numeric(série, errors="coerce")	Convertit en numérique
pd.to_numeric(série, errors="ignore")	Laisse tel quel si erreur
pd.to_numeric(série, downcast="integer")	Downcast vers entier minimal
pd.to_datetime(série)	Convertit en datetime
pd.to_timedelta(série)	Convertit en timedelta
pd.to_pickle(obj, "f.pkl")	Séréalise un objet
pd.unique(série)	Valeurs uniques (plus rapide que .unique())
pd.isna(valeur)	Teste si NaN / NaT / None
pd.notna(valeur)	Teste si non-nul
pd.isnull(valeur)	Identique à isna
pd.notnull(valeur)	Identique à notna
pd.concat(liste_df)	Concatène des DataFrames
pd.merge(df1, df2)	Fusionne deux DataFrames
pd.pivot(df)	Pivote un DataFrame
pd.pivot_table(df)	Tableau croisé dynamique
pd.melt(df)	Format long
pd.crosstab(a, b)	Tableau croisé de fréquences
pd.cut(série, bins)	Discretisation par intervalles
pd.qcut(série, q)	Discretisation par quantiles
pd.factorize(série)	Encodage numérique
pd.get_dummies(df)	One-hot encoding

<code>pd.from_dummies(df)</code>	Inverse one-hot
<code>pd.json_normalize(data)</code>	Normalise un JSON imbriqué
<code>pd.lreshape(df, groups)</code>	Reshape format long
<code>pd.wide_to_long(df, stubnames, i, j)</code>	Large → long
<code>pd.eval(expression)</code>	Évalue une expression string
<code>pd.eval(expression, target=df)</code>	Évalue et stocke dans un df
<code>pd.set_option("display.max_rows", 100)</code>	Configure l'affichage
<code>pd.set_option("display.max_columns", None)</code>	Affiche toutes les colonnes
<code>pd.set_option("display.width", None)</code>	Largeur d'affichage automatique
<code>pd.set_option("display.float_format", "{:.2f}".format)</code>	Format flottants
<code>pd.set_option("display.max_colwidth", None)</code>	Largeur max des colonnes texte
<code>pd.set_option("mode.chained_assignment", None)</code>	Désactive l'avertissement
<code>pd.get_option("display.max_rows")</code>	Lit une option
<code>pd.reset_option("display.max_rows")</code>	Remet l'option par défaut
<code>pd.reset_option("all")</code>	Remet toutes les options par défaut
<code>pd.describe_option("display.max_rows")</code>	Décrit une option
<code>pd.option_context("display.max_rows", 50)</code>	Contexte d'option temporaire
<code>pd.options.display.max_rows = 100</code>	Accès direct aux options
<code>pd.show_versions()</code>	Affiche les versions installées
<code>pd.infer_freq(index)</code>	Infère la fréquence d'un index
<code>pd.offsets.Day(1)</code>	Offset d'un jour
<code>pd.offsets.MonthEnd()</code>	Fin de mois
<code>pd.offsets.BusinessDay()</code>	Jour ouvrable
<code>pd.testing.assert_frame_equal(df1, df2)</code>	Teste l'égalité (tests unitaires)
<code>pd.testing.assert_frame_equal(df1, df2, check_exact=False, rtol=1e-5)</code>	Test avec tolérance numérique
<code>pd.testing.assert_series_equal(s1, s2)</code>	Teste l'égalité de Series
<code>pd.testing.assert_index_equal(i1, i2)</code>	Teste l'égalité d'index
<code>pd.array(data, dtype)</code>	Crée un ExtensionArray
<code>pd.api.types.is_numeric_dtype(df["col"])</code>	Teste si type numérique
<code>pd.api.types.is_string_dtype(df["col"])</code>	Teste si type chaîne
<code>pd.api.types.is_datetime64_dtype(df["col"])</code>	Teste si type datetime
<code>pd.api.types.is_categorical_dtype(df["col"])</code>	Teste si type catégoriel

EXEMPLES :

```

pd.set_option("display.max_columns", None)
with pd.option_context("display.float_format", "{:.2f}".format):
    print(df)
df["id"] = pd.to_numeric(df["id"], errors="coerce")
data = pd.json_normalize(json_data, record_path="items", meta=["date", "source"])

```

RÉCAPITULATIF DES FORMATS DE FICHIERS SUPPORTÉS PAR PANDAS

FORMAT	LECTURE	ÉCRITURE	NOTES
CSV	<code>pd.read_csv()</code>	<code>df.to_csv()</code>	Format universel
TSV	<code>pd.read_table()</code>	<code>df.to_csv(sep="\t")</code>	Tabulation comme sépar.
FWF	<code>pd.read_fwf()</code>	—	Largeur fixe
Excel .xlsx	<code>pd.read_excel()</code>	<code>df.to_excel()</code>	Multi-feuilles via ExcelWriter
Excel .xls	<code>pd.read_excel()</code>	<code>df.to_excel()</code>	Ancien format Excel
Excel .ods	<code>pd.read_excel()</code>	<code>df.to_excel()</code>	OpenDocument Spreadsheet
JSON	<code>pd.read_json()</code>	<code>df.to_json()</code>	Multiples orientations
JSONL	<code>pd.read_json(lines=T)</code>	<code>df.to_json(lines=T)</code>	Une ligne = un objet
JSON imbriq	<code>pd.json_normalize()</code>	—	Normalise les sous-objets
Parquet	<code>pd.read_parquet()</code>	<code>df.to_parquet()</code>	Colonnaire, compressé
Feather	<code>pd.read_feather()</code>	<code>df.to_feather()</code>	Échange rapide Python/R
HDF5	<code>pd.read_hdf()</code>	<code>df.to_hdf()</code>	Requêttable, gros volumes
Pickle	<code>pd.read_pickle()</code>	<code>df.to_pickle()</code>	Sérialisation Python
SQL	<code>pd.read_sql()</code>	<code>df.to_sql()</code>	Toutes bases via connexion
HTML	<code>pd.read_html()</code>	<code>df.to_html()</code>	Tableaux web
XML	<code>pd.read_xml()</code>	<code>df.to_xml()</code>	Données structurées
Stata .dta	<code>pd.read_stata()</code>	<code>df.to_stata()</code>	Logiciel statistique
SAS	<code>pd.read_sas()</code>	—	Logiciel SAS
SPSS .sav	<code>pd.read_spss()</code>	—	Logiciel SPSS
ORC	<code>pd.read_orc()</code>	<code>df.to_orc()</code>	Colonnaire (Big Data)
Clipboard	<code>pd.read_clipboard()</code>	<code>df.to_clipboard()</code>	Presse-papiers système
Markdown	—	<code>df.to_markdown()</code>	Retourne une chaîne
LaTeX	—	<code>df.to_latex()</code>	Retourne une chaîne
dict Python	—	<code>df.to_dict()</code>	Vers structure Python
numpy array	—	<code>df.to_numpy()</code>	Vers array NumPy

— RÉCAPITULATIF ExcelWriter
USE CASE

SOLUTION

1 df → 1 feuille	df.to_excel("f.xlsx")
N df → N feuilles	pd.ExcelWriter + .to_excel() dans with
Ajouter une feuille à un fichier existant	pd.ExcelWriter(mode="a")
Remplacer une feuille existante	pd.ExcelWriter(if_sheet_exists="replace")
Plusieurs df sur la même feuille	to_excel(writer, startrow=..., startcol=...)
Une feuille par groupe	groupby() + boucle + ExcelWriter
Figier lignes/colonnes	to_excel(freeze_panes=(1,0))
Lire toutes les feuilles	pd.read_excel(sheet_name=None) → dict

=====